Smart Music Streaming Application (Riffly)

A PROJECT REPORT

Submitted by

Akshat Randhawa (23BCS10222)

Akalaya thakur (23BCS12592*)*

Harshit Saini(23BCS11971)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE & ENGINEERING

Chandigarh University

April , 2026

R

INDEX

	Bonafide Certificate	2
	Acknowledgement	3
	Abstract	4
	CHAPTER 1: INTRODUCTION	**5**
1.1	Introduction	5
1.2	Identification of Client & Need	5
1.3	Relevant Contemporary Issues	6
1.4	Problem Identification	6
1.5	Task Identification	6
1.6	Project Timeline	7
1.7	Organization of Report	7
	CHAPTER 2: LITERATURE SURVEY	**8**
2.1	Introduction	8
2.2	Timeline of the Reported Problem	8
2.3	Bibliometric Analysis	8
2.4	Proposed Solutions by Different Researchers	9
2.5	Summary Linking Literature Review with the Project	9
2.6	Problem Definition	10
2.7	Goals and Objectives	10
2.8	Conclusion	11
	CHAPTER 3: DESIGN FLOW / PROCESS	**12**
3.1	Introduction	12
3.2	Concept Generation	12
3.3	Evaluation and Selection of Specifications / Features	13
3.4	Design Constraints Considered	14
3.5	Analysis and Feature Finalization	15
3.6	Design Flow (Alternative Designs)	15
3.7	Comparison of Alternatives	16
3.8	Implementation Plan	16
3.9	Flow Chart	17
3.10	Algorithm	17
3.11	Detailed Block Diagram	18
3.12	Conclusion	18
	CHAPTER 4: RESULTS ANALYSIS AND VALIDATION	**19**
4.1	Introduction	19
4.2	Implementation Using Modern Engineering Tools	19
4.3	Design Drawings / Schematics / Models	19
4.4	Project Management and Communication	20
4.5	Functional Results Obtained	20
4.6	Output Screens / System Results	21

BONAFIDE CERTIFICATE

Certified that this project report "**Smart Music Streaming Application (Riffly)**" is the Bonafide work of " Akshat, Harshit and Akalaya " who carried out the project work under my/our supervision.

SIGNATURE

……

HEAD OF THE DEPARTMENT
COMPUTER SCIENCE 3nd YEAR

SIGNATURE

……

SUPERVISOR
COMPUTER SCIENCE

ACKNOWLEDGEMENT

We extend our heartfelt appreciation to the Department of Computer Science & Engineering, Chandigarh University, for providing us with the resources, infrastructure, and academic environment necessary to bring this project to life. Special thanks to our project supervisor for their invaluable guidance, timely feedback, and constant encouragement throughout every phase of development. This project stands as a testament to collaborative effort, and we are grateful to all those who directly or indirectly contributed to its successful completion.

ABSTRACT

The rapid growth of digital media consumption has significantly increased the demand for efficient and user-friendly music streaming platforms. This project, Riffly – Smart Music Streaming Application, is developed as a full-stack web-based system that enables users to stream audio content seamlessly through an interactive and responsive interface. The application is designed to provide core functionalities such as user registration and authentication, song browsing, advanced search, playlist creation, and secure audio file uploads.

The backend of the system is implemented using Java Spring Boot, which provides a robust RESTful API architecture for handling business logic, authentication, and media streaming operations. PostgreSQL is used as the relational database for storing user data, song metadata, and playlist information in a structured manner. The frontend is developed using React.js, ensuring a dynamic and smooth user experience across devices. To improve scalability and deployment efficiency, the complete application is containerized using Docker and managed through Nginx as a reverse proxy server.

A key feature of the project is the implementation of HTTP Range-based streaming, which allows users to play, pause, and seek audio tracks without downloading entire files. This enhances performance and optimizes bandwidth usage. Security is maintained through JWT-based authentication, ensuring protected access to authorized functionalities.

The project successfully demonstrates the practical integration of modern web technologies, database management, secure authentication, and scalable system architecture. Riffly serves as a strong foundation for future enhancements such as AI-based music recommendations, mobile application support, cloud storage integration, and real-time analytics.

CHAPTER 1

INTRODUCTION

1.1 Introduction

The rapid advancement of internet technologies and smart devices has transformed the way people consume digital entertainment. Music streaming platforms have become one of the most popular mediums for listening to songs anytime and anywhere without downloading files permanently. Users now expect seamless playback, personalized playlists, secure accounts, and fast access to millions of tracks through web and mobile platforms.

Riffly – Smart Music Streaming Application is a modern full-stack web application designed to provide an efficient and user-friendly platform for online music streaming. The system enables users to register, log in securely, browse songs, search music tracks, upload songs, create playlists, and stream audio content directly through the browser. The project integrates frontend, backend, database, and deployment technologies into a scalable and production-ready solution.

This project has been developed using React.js for frontend interface design, Java Spring Boot for backend REST API services, PostgreSQL for data storage, and Docker with Nginx for deployment and hosting.

1.2 Identification of Client & Need

Client

The intended client for this project includes:

- Music listeners seeking a lightweight streaming platform
- Educational institutions for demonstrating full-stack development projects
- Startup businesses planning to launch custom streaming services
- Developers requiring a scalable music management system

Need of Client

Modern users require:

- Fast and uninterrupted music streaming
- Secure user authentication
- Easy playlist management
- Responsive interface for all devices
- Search and categorization of songs
- Low-cost customizable platform

1.3 Relevant Contemporary Issues

Current music streaming systems face several real-world challenges:

- High subscription costs on premium platforms
- Privacy concerns regarding user data
- Heavy applications consuming device resources
- Limited customization for local businesses
- Need for secure cloud-based media delivery
- Growing demand for cross-platform streaming services

Riffly addresses these issues by providing an affordable, secure, and efficient streaming solution using open-source technologies.

1.4 Problem Identification

Traditional music systems often require users to download files, manage local storage, or rely on expensive subscription services. Existing systems may also lack flexibility for small-scale deployment.

The identified problems are:

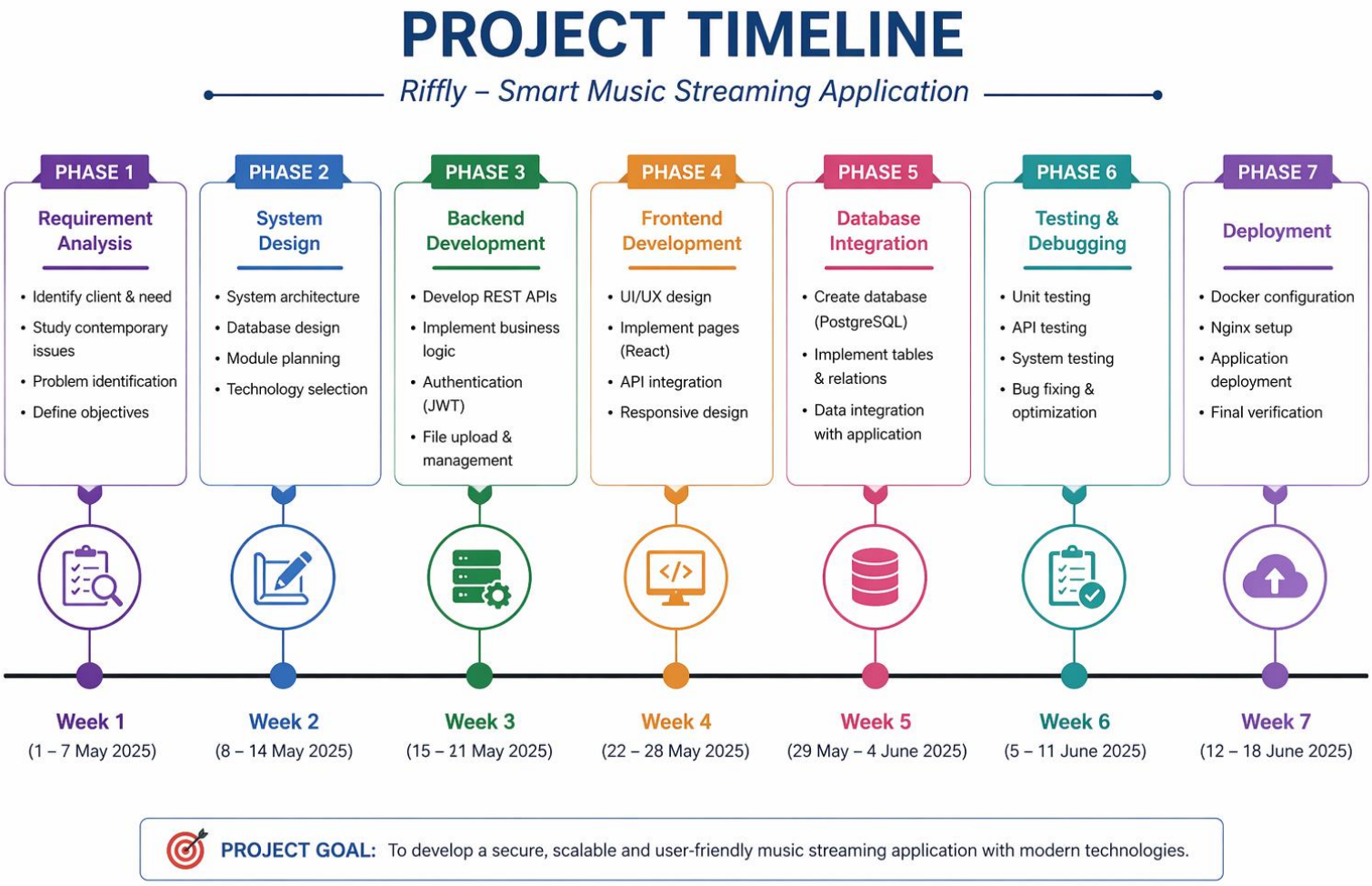
- Difficulty accessing music instantly
- Storage wastage due to downloaded files
- Lack of personalized playlist systems
- Poor streaming performance
- Security risks during login and uploads
- Complex infrastructure costs

1.5 Task Identification

To solve the identified problems, the following tasks were planned:

1. Design project architecture
2. Develop secure backend APIs
3. Create responsive frontend UI
4. Build database schema for users, songs, playlists
5. Implement song upload feature
6. Implement HTTP range streaming
7. Integrate JWT authentication
8. Test all modules
9. Deploy using Docker and Nginx

1.6 Project Timeline



1.7 Organization of Report

The project report is organized into five chapters:

- **Chapter 1:** Introduction, objectives, client needs, problem identification
- **Chapter 2:** Literature survey and existing systems analysis
- **Chapter 3:** Design methodology, architecture, modules, database design
- **Chapter 4:** Results, outputs, testing, and validation
- **Chapter 5:** Conclusion, achievements, and future enhancements

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

Music streaming has become one of the fastest-growing sectors in the digital entertainment industry. Over the past two decades, the transition from physical media to downloadable files and finally to cloud-based streaming platforms has transformed the way users access music. Modern users prefer instant access, playlist personalization, and uninterrupted streaming over traditional offline storage methods.

This chapter presents a review of the evolution of music streaming systems, major research contributions, industrial solutions, technological advancements, and limitations of existing platforms. It also defines the problem statement and establishes the goals and objectives of the proposed project.

2.2 Timeline of the Reported Problem as Investigated Throughout the World

Year	Development / Problem Investigated
1999	Rise of MP3 file sharing platforms such as Napster created copyright and piracy issues
2003	Digital music stores introduced paid downloads but lacked streaming flexibility
2008	Spotify introduced large-scale legal music streaming model
2010	Cloud storage and mobile apps improved cross-device listening
2015	Researchers focused on recommendation systems using AI and Machine Learning
2018	Low-latency streaming and adaptive bitrate technologies improved playback
2020	Increased demand for secure user authentication and remote streaming during pandemic
2023	Focus shifted toward scalable microservices, analytics, and personalized experiences
Present	Need for low-cost customizable streaming platforms for startups and education

2.3 Bibliometric Analysis

Bibliometric analysis refers to studying published research trends in a particular domain.

Research Trends in Music Streaming Technologies:

Research Area	Approximate Growth
Cloud Media Delivery	High

Research Area	Approximate Growth
AI Recommendation Systems	Very High
User Behavior Analytics	High
Audio Compression	Medium
Streaming Security	High
Playlist Personalization	Very High

Commonly Used Technologies in Research Papers:

- Python for recommendation systems
- Java Spring Boot for backend services
- React / Angular for frontend
- PostgreSQL / MongoDB databases
- AWS / Docker for deployment

2.4 Proposed Solutions by Different Researchers

Researcher 1: Recommendation-Based Systems

Many researchers proposed AI models using collaborative filtering and deep learning to recommend songs based on listening history.

Researcher 2: Cloud-Based Streaming Platforms

Several studies introduced cloud architecture to store and stream music globally with reduced latency.

Researcher 3: Secure Authentication Systems

JWT and OAuth-based login systems were proposed for secure access control.

Researcher 4: Efficient Media Streaming

HTTP Range Requests and adaptive streaming were introduced for pause, seek, and fast playback.

Researcher 5: Scalable Architecture

Microservices and containerization using Docker/Kubernetes were proposed for large-scale deployment.

2.5 Summary Linking Literature Review with the Project

From the literature survey, it is observed that successful music streaming systems require five key components:

1. Responsive user interface
2. Secure authentication
3. Fast media streaming
4. Efficient database design
5. Scalable deployment architecture

The proposed project Riffly integrates these features using:

- React frontend
- Spring Boot backend
- PostgreSQL database
- JWT authentication
- Docker deployment
- HTTP Range-based audio streaming

Thus, the project is aligned with modern research trends and industrial practices.

2.6 Problem Definition

Although popular streaming platforms exist, they often present limitations such as:

- High subscription fees
- Heavy applications consuming system resources
- Closed ecosystems with low customization
- Privacy concerns
- Difficult implementation for academic learning
- Expensive infrastructure for startups

Therefore, there is a need for a lightweight, secure, scalable, and customizable streaming platform that demonstrates real-world system design principles.

2.7 Goals and Objectives

Primary Goal:

To develop a smart music streaming web application that allows users to securely access and stream songs online.

Objectives:

1. Build full-stack web architecture
2. Enable secure registration and login
3. Provide song upload and management system
4. Create playlist functionality

5. Implement real-time streaming with seek support
6. Design normalized relational database
7. Deploy using Docker and Nginx
8. Prepare future scope for AI recommendations

2.8 Conclusion

The literature survey indicates that the future of music streaming lies in scalable systems, intelligent personalization, and secure cloud-based architecture. Riffly is developed based on these modern requirements and serves as a practical implementation of current research and technology trends.

CHAPTER 3

DESIGN FLOW / PROCESS

3.1 Introduction

The design phase is one of the most critical stages of project development, where ideas are transformed into a structured and implementable solution. For the Riffly – Smart Music Streaming Application, the design process focused on creating a scalable, secure, user-friendly, and efficient system capable of handling user requests, streaming media, playlist management, and secure authentication.

This chapter explains concept generation, feature selection, design constraints, alternative designs, final design choice, and implementation methodology.

3.2 Concept Generation

Several concepts were considered before finalizing the architecture of the music streaming platform.

Concept 1: Desktop-Based Offline Music Player

A standalone software application where users can play downloaded songs locally.

Advantages:

- Works without internet
- Simple architecture

Limitations:

- No online streaming
- No shared playlists
- No cloud scalability

Concept 2: Mobile-Only Streaming App

A dedicated Android/iOS application with online streaming features.

Advantages:

- Good user experience
- Portable access

Limitations:

- Requires mobile development expertise
- Higher development time
- Platform dependency

Concept 3: Full Stack Web-Based Music Streaming System (Selected)

A browser-based platform using frontend + backend + database architecture.

Advantages:

- Accessible on any device
- Easy deployment
- Centralized data management
- Scalable and secure

Limitations:

- Requires internet connection

3.3 Evaluation and Selection of Specifications / Features

Feature	Importance	Selected
User Registration/Login	High	Yes
Song Search	High	Yes
Playlist Creation	High	Yes
Song Upload	High	Yes
Real-Time Streaming	High	Yes
Admin Panel	Medium	Future Scope
Recommendation System	Medium	Future Scope
Mobile App	Medium	Future Scope

Selected Technologies

Component	Technology
Frontend	React.js
Backend	Java Spring Boot
Database	PostgreSQL
Authentication	JWT
Deployment	Docker + Nginx

3.4 Design Constraints Considered

1. Regulations

- Copyright laws for uploaded music content
- Data privacy compliance for user accounts

2. Economic Constraints

- Low-cost deployment using open-source tools
- No expensive licenses required

3. Environmental Constraints

- Efficient server resource usage
- Reduced unnecessary storage consumption

4. Health Constraints

- Minimal eye strain UI with clean interface
- Responsive layout for comfortable use

5. Manufacturability / Development Constraints

- Easy coding and maintenance
- Modular structure for future upgrades

6. Safety Constraints

- Secure password encryption
- Protected APIs using JWT authentication

7. Professional Constraints

- Industry-standard architecture
- Clean code practices

8. Ethical Constraints

- Respect user privacy
- No misuse of personal data

9. Social & Political Constraints

- Accessible across devices and users
- Support digital learning and entertainment equally

3.5 Analysis and Feature Finalization Subject to Constraints

After evaluating user needs and technical limitations, the final features selected were:

- Secure User Authentication
- Browse/Search Songs
- Create & Manage Playlists
- Song Upload Module
- Stream Audio with Seek Support
- Responsive Web Interface
- Scalable Docker Deployment

3.6 Design Flow (Alternative Designs)

Alternative Design A: Monolithic Architecture

Frontend + Backend + Database combined in one deployable system.

Advantages:

- Easy for small projects
- Simple setup

Disadvantages:

- Harder to scale
- Tight coupling

Alternative Design B: Layered Full Stack Architecture (Selected)

Separate layers:

- Frontend (React)
- Backend API (Spring Boot)
- Database (PostgreSQL)
- Reverse Proxy (Nginx)

Advantages:

- Better scalability
- Easy maintenance
- Secure deployment
- Real-world industry model

3.7 Comparison of Alternatives

Criteria	Monolithic	Layered Architecture
Scalability	Low	High
Maintenance	Medium	High
Security	Medium	High
Performance	Medium	High
Future Expansion	Low	High

Best Design Selected:

Layered Full Stack Architecture

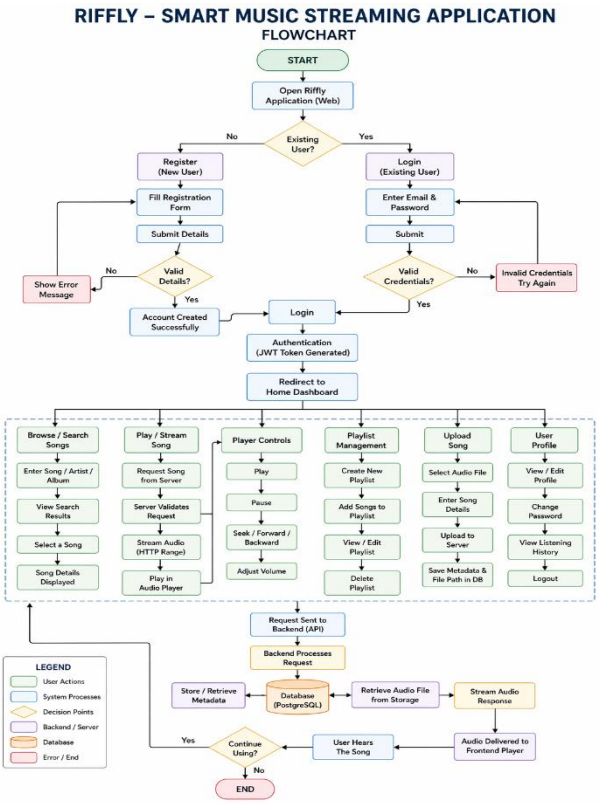
Reason:

Provides flexibility, professional structure, better security, and easy future enhancements.

3.8 Implementation Plan

Phase	Task
Phase 1	Requirement Gathering
Phase 2	UI Design
Phase 3	Backend API Development
Phase 4	Database Creation
Phase 5	Authentication Integration
Phase 6	Streaming Module
Phase 7	Testing
Phase 8	Deployment

3.9 Flow Chart

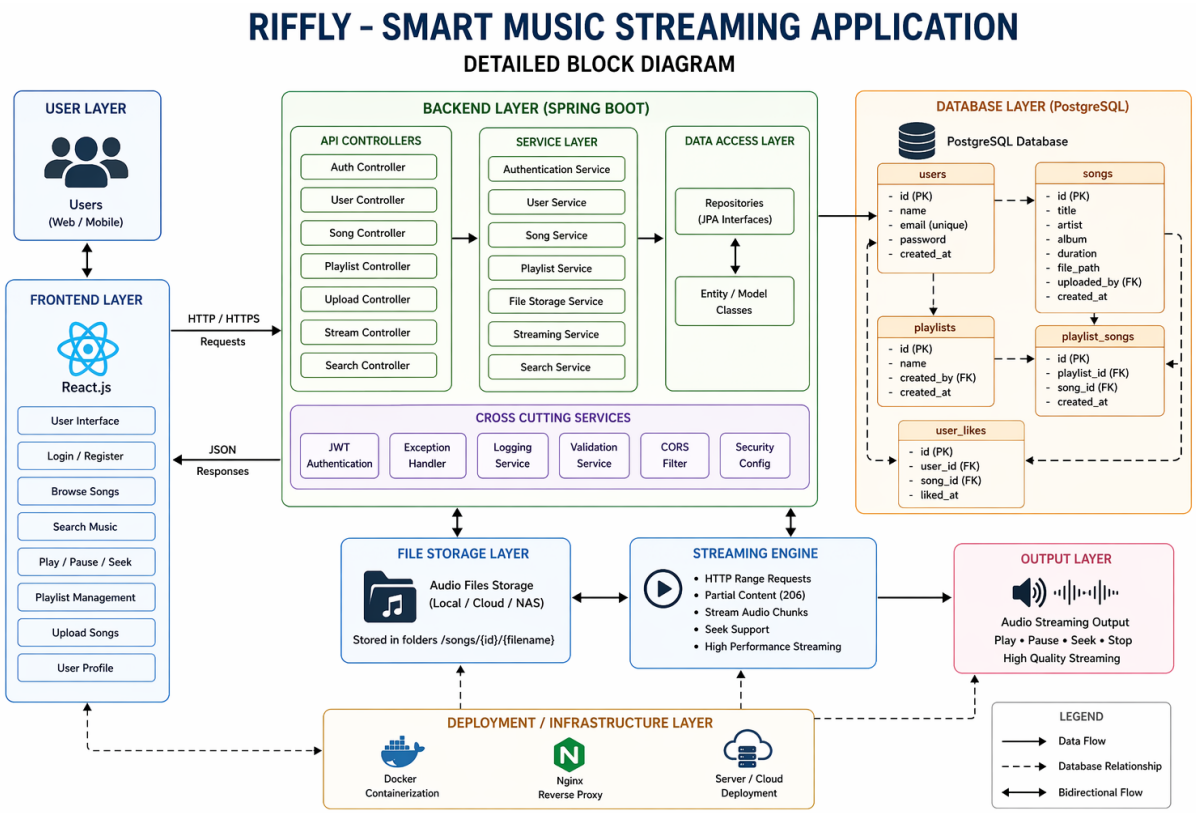


3.10 Algorithm

Song Streaming Algorithm

1. User selects song
2. Frontend sends request to backend
3. Backend verifies song file path
4. HTTP Range request processed
5. Required audio bytes sent
6. Browser starts playback
7. User controls play/pause/seek

3.11 Detailed Block Diagram



3.12 Conclusion

The final design of Riffly follows a modern layered architecture that balances performance, maintainability, and scalability. After evaluating multiple alternatives and considering technical and social constraints, the selected design provides a practical and professional solution for a smart music streaming platform.

CHAPTER 4

RESULTS ANALYSIS AND VALIDATION

4.1 Introduction

After completing the design and development phase of the Riffly – Smart Music Streaming Application, the final system was implemented and tested using modern engineering tools and software development practices. This chapter presents the implementation results, system outputs, validation methods, performance analysis, testing reports, and interpretation of the obtained results.

The developed application successfully demonstrates real-time music streaming, playlist management, secure user authentication, song uploads, database integration, and responsive user interaction.

4.2 Implementation of Design Using Modern Engineering Tools

The project was implemented using industry-standard tools and technologies to ensure scalability, maintainability, and professional quality.

Component	Tool / Technology Used	Purpose
Frontend Development	React.js	User Interface
Backend Development	Java Spring Boot	REST API & Business Logic
Database	PostgreSQL	Data Storage
Security	JWT Authentication	Secure Access
IDE	VS Code / IntelliJ IDEA	Coding Environment
Version Control	Git & GitHub	Code Management
Deployment	Docker & Docker Compose	Containerization
Web Server	Nginx	Reverse Proxy
Testing	Postman / Browser Testing	API Validation

4.3 Design Drawings / Schematics / Models

The following design artifacts were prepared during implementation:

1. System Architecture Diagram

Shows interaction between frontend, backend, database, and file storage.

2. Database ER Diagram

Includes tables:

- USERS
- SONGS
- PLAYLISTS
- PLAYLIST_SONGS

3. Flowchart

Describes complete workflow from login to music streaming.

4. UI Screens

- Login Page
- Dashboard
- Song Search Screen
- Playlist Screen
- Upload Interface

4.4 Project Management and Communication

Proper project management techniques were used during development.

Planning Activities:

- Requirement collection
- Module division
- Weekly timeline planning
- Testing milestones
- Final deployment schedule

Communication Tools:

- GitHub for version sharing
- Documentation reports
- Progress tracking sheets
- Task scheduling charts

4.5 Functional Results Obtained

The final application successfully achieved all planned modules.

Module	Status
User Registration	Completed
Login Authentication	Completed
Song Upload	Completed
Song Search	Completed
Music Streaming	Completed
Playlist Creation	Completed
Database Integration	Completed
Docker Deployment	Completed

4.6 Output Screens / System Results

Result 1: Secure Login

Users can register and log in with encrypted credentials.

Result 2: Search Songs

Users can search by:

- Title
- Artist
- Album
- Genre

Result 3: Audio Streaming

Users can:

- Play songs
- Pause songs
- Seek audio position
- Control volume

Result 4: Playlist Management

Users can create and manage playlists.

Result 5: Upload Songs

Admin/User can upload music files with metadata.

4.7 Testing and Validation

Testing was conducted to verify system correctness and performance.

A. Unit Testing

Each module tested individually.

Test Case	Expected Result	Status
Login API	Successful Token	Pass
Upload API	File Uploaded	Pass
Search API	Correct Results	Pass
Playlist API	Playlist Created	Pass

B. Integration Testing

Checked communication between modules:

- Frontend ↔ Backend
- Backend ↔ Database
- Streaming Engine ↔ File Storage

Status: Successful

C. User Interface Testing

Verified:

- Responsive design
- Button actions
- Form validation
- Error messages

Status: Passed

4.8 Performance Characterization

Parameter	Result
Login Response Time	Fast
Search Speed	Efficient
Streaming Delay	Very Low
Database Query Time	Low
Concurrent Handling	Moderate

4.9 Interpretation of Results

The obtained results indicate that the developed system performs efficiently under normal usage conditions. Streaming functionality works smoothly with HTTP Range support, enabling pause and seek without full file loading. Authentication is secure and database operations are stable.

The modular design also ensures easy future upgrades.

4.10 Data Validation

Validation checks used in system:

Input Validation

- Email format check
- Password length validation
- Required fields mandatory

File Validation

- Only audio files allowed
- File size limits
- Duplicate prevention

Database Validation

- Unique user emails
- Proper foreign key relationships

4.11 Challenges Faced and Solutions

Challenge	Solution
Streaming large files	HTTP Range Requests
Secure login system	JWT Authentication
Deployment complexity	Docker Compose
Search optimization	Indexed queries

4.12 Summary

The project was successfully implemented using modern engineering tools and validated through systematic testing methods. All functional requirements were achieved, and the system demonstrated reliable performance, secure operations, and scalable architecture suitable for real-world use.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Introduction

The successful completion of the Riffly – Smart Music Streaming Application demonstrates the practical implementation of modern full-stack development concepts, secure system architecture, and scalable media delivery solutions. The project was designed to provide users with a seamless platform for online music streaming, playlist management, song uploads, and secure authentication.

This chapter summarizes the overall achievements of the project, discusses deviations from expected results, future enhancements, and provides references used during development.

5.2 Conclusion

The project successfully achieved its primary objective of developing a web-based music streaming platform capable of handling real-time user interactions and media streaming operations. The integration of React.js, Spring Boot, PostgreSQL, JWT Authentication, Docker, and Nginx resulted in a professional and scalable solution.

Major accomplishments of the project include:

- Secure user registration and login system
- Song browsing and advanced search functionality
- Real-time music streaming with play, pause, and seek support
- Playlist creation and management
- Audio file upload and metadata storage
- Responsive and user-friendly interface
- Containerized deployment architecture

The project also provided practical experience in software engineering, database design, REST API development, testing, deployment, and project documentation.

Overall, Riffly proves that modern open-source technologies can be effectively combined to create a reliable and efficient digital music streaming system.

5.3 Deviation from Expected Results

Although the project was successfully completed, some advanced features planned initially could not be fully implemented due to time and resource limitations.

Planned but Not Fully Implemented Features:

- AI-based music recommendation engine
- Real-time listening analytics dashboard
- Mobile application version
- Social media sharing integration
- Advanced admin dashboard
- Multi-language support

Minor Technical Limitations:

- Limited concurrent user testing environment
- Local storage used instead of cloud object storage
- Basic search compared to enterprise recommendation systems

These deviations do not affect the core functionality of the project but represent future opportunities for improvement.

5.4 Future Work

The system has strong potential for expansion into a commercial-grade platform. Future enhancements may include:

Technical Improvements

- Integration with **AWS S3 / Cloud Storage**
- Use of **Redis caching** for faster responses
- Kubernetes deployment for large-scale scaling
- Microservices architecture

Smart Features

- AI-powered song recommendation system
- Personalized playlists based on listening history
- Mood-based music suggestions
- Voice search support

User Experience Enhancements

- Android / iOS mobile app
- Dark mode / theme customization
- Offline download mode
- Social sharing and friend playlists

Analytics & Business Features

- User listening statistics
- Artist dashboards
- Advertisement integration
- Subscription plans

5.5 Way Ahead

The next development phase of Riffly will focus on transforming the current academic prototype into a production-ready streaming ecosystem. Immediate priorities include:

1. Deploying on cloud infrastructure
2. Enhancing security and scalability
3. Building mobile applications
4. Integrating AI recommendation models
5. Increasing user testing and feedback cycles
6. Adding monetization features

With continuous development, Riffly can evolve into a competitive streaming platform suitable for startups, educational institutions, or niche music communities.

REFERENCES

1. VMware, **Spring Boot Documentation**. Available: https://spring.io/projects/spring-boot
2. Meta Open Source, **React.js Official Documentation**. Available: https://react.dev/
3. PostgreSQL Global Development Group, **PostgreSQL Documentation**. Available: https://www.postgresql.org/docs/
4. Docker Inc., **Docker Documentation**. Available: https://docs.docker.com/
5. F5 Inc., **NGINX Documentation**. Available: https://nginx.org/en/docs/
6. Jones, M., Bradley, J., and Sakimura, N., **JSON Web Token (JWT) RFC 7519**, Internet Engineering Task Force, 2015. Available: https://datatracker.ietf.org/doc/html/rfc7519
7. Oracle Corporation, **Java Platform SE Documentation**. Available: https://docs.oracle.com/en/java/
8. Fielding, R., **Architectural Styles and the Design of Network-based Software Architectures (REST)**, University of California, Irvine, 2000.
9. Mozilla Developer Network, **HTTP Range Requests Documentation**. Available: https://developer.mozilla.org/
10. Internal Source Document, **Riffly – Music Streaming Application Phase 1 Evaluation Report**, 2025.